Exercises on Arrays - Sorting and Searching

Computer Programming 1

<2026-10-28>

Outline

Searching: Linear

Write a function which searches for an alement in an (unsorted) Array. The function returns the index in the array.

- Variations: implement with a `break` at the first occurrence; implement with a boolean flag.

Searching: Multiple Occurrences

Assume an element can occur more than once in an Array and no more than 10 times.

Write a function which return all the indexes in which the element occurs. Use an array to store the matching indexes

```
int a = { 1, 1, 2, 1, 1, 2 };
int found[10];

search(1, a) /* -> 0, 1, 3, 4 */
/* search does a side-effect on found and stores the indexes as follows: [0, 1, 3, 4] */
```

Searching - Binary Search

If an array `A` is sorted, then you can search efficiently for an element `EL` as follows (binary search):

- Compare `EL` with the element in the middle of the array (let us say the element at index `i_middle`)
- If `EL` is equal to `A[i_middle]`, you are done
- Otherwise:
  - If `EL` is bigger, you search for `EL` in the right portion of the array (in the array from `i_middle + 1` to the end of the array
  - If `EL` is lower, you seach for `EL` in the lower portion of the array (in the array from `0` to `i_middle - 1`

- Implement the binary search using a recursive function. Define a function which takes four arguments, the element to find, the array, the minimum index in which to search, the maximum index in which to search

Sorting - Selection Sort

(Re-)implement the selection sort.

Sorting - Insertion Sort

(Re-)implement the insertion sort.

Sorting - Bubble Sort

(Re-)implement the bubble sort.

Check for Sortedness

Create an array with dimension `N` and let the user fill it from the standard input.

Then write a function `isSorted(arr, index, size)` to determine if the array is sorted in non-decreasing order.

Hint:

1. Loop over the array and break if ...
2. (inefficient) sort the array and compare with the original

Variation: Recursive Check for Sortedness

Implement the previous exercise in a recursive manner, taking inspiration from QuickSort.